

1 Overview

The assignment involves:

1. Setting up a **TurtleBot3 house environment** in ROS 2.
2. Implementing a Behaviour Tree (BT) or Finite State Machine (FSM) to implement acting behaviours for room inspection and energy management behaviours.
3. Creating a PDDL domain and problem for planning the robot's behaviour.
4. Using PlanSys2 to create a plan and execute it.

The marks are allocated 20% for Step 1, 30% for Step 2, 30% for Step 3, and 20% for Step 4. Highest marks will be achieved by good quality, well-commented code that accurately and efficiently implements the required behaviour and demonstrates effective use of both acting (Behaviour Trees or FSM) and planning.

Carefully read the submission instructions in section 3 also.

2 Project Description

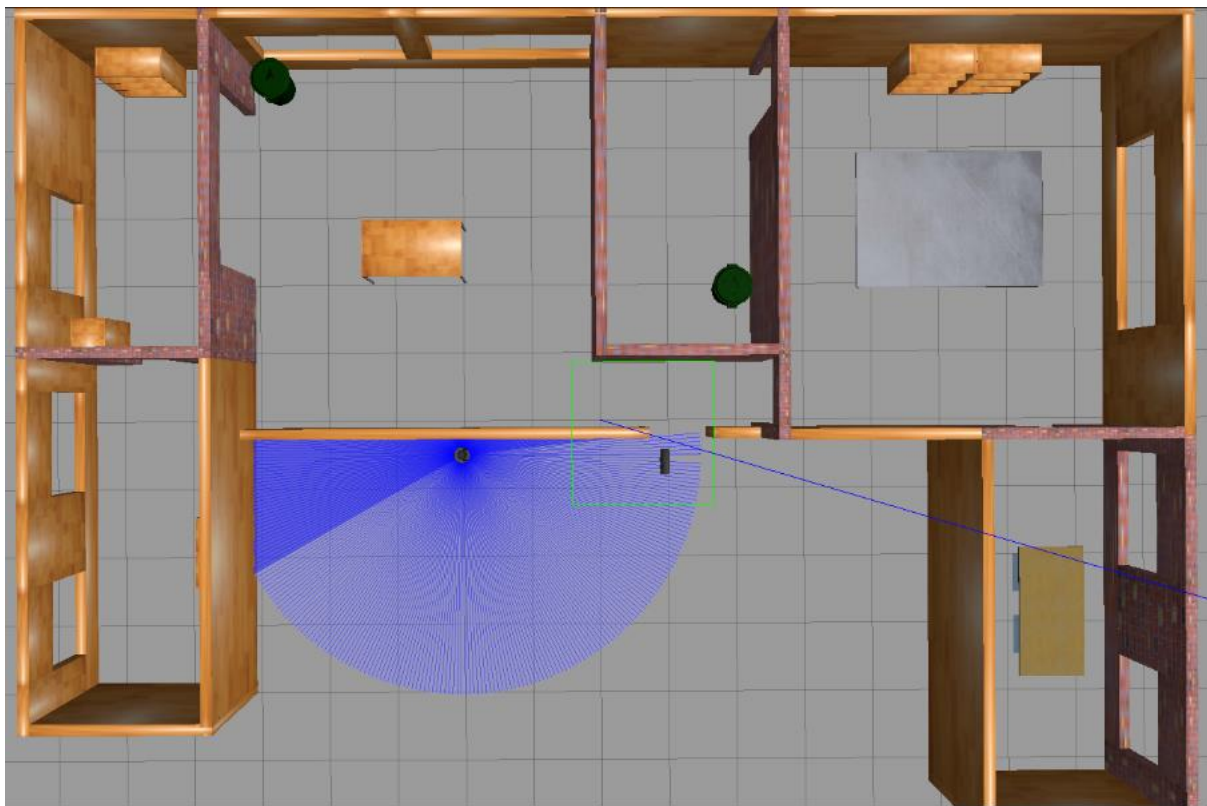


Figure 1 Revised Simulation environment for assignment

You are required to implement both planning and acting behaviours for an autonomous robot in a simulated indoor TurtleBot3 house environment, shown in Figure 1. The objective is to patrol the environment and manage lighting and energy usage efficiently.

You must predefine at least five named/numbered waypoints in this environment (e.g., living room, kitchen, bedroom, hallway etc.). Waypoints can be obtained by teleoperating

Assignment – Smart Energy Management Robot

the robot and recording its pose using *ros2 topic echo /amcl_pose*. The robot will perform inspection checks at these waypoints, so their locations should maximise coverage of all rooms.

To perform an inspection, the robot should execute a room state check at each waypoint. Room occupancy should be simulated as a state (occupied/unoccupied), which may change over time (e.g., every 30–60 seconds) or be determined when the robot arrives at a waypoint (e.g., by randomly assigning occupied/unoccupied during inspection). If a room is occupied, the robot must ensure the light is ON; otherwise, it must ensure the light is OFF. Energy usage should also be simulated, for example by triggering high energy when lights remain ON in unoccupied rooms, which the robot must then reduce.

You must define a planning domain (in PDDL) and use PlanSys2 to specify a planning problem (using the PlanSys2 API) and create and execute a plan that determines the order to visit and inspect the waypoints.

At the outset of the simulation, two waypoints have active energy priorities. At the start of each simulation run:

1. your code must randomly select one of your waypoints as the *critical_energy* waypoint and another as the *high_energy* waypoint. The generated plan must ensure that *critical_energy* waypoints are handled before *high_energy* waypoints, which are handled before all other waypoints. Visiting a lower-priority waypoint before a higher-priority one is incorrect behaviour.
2. Your code must also choose a random location for the robot as the starting point.

Navigation between waypoints is performed using the Nav2 stack with real-time obstacle avoidance. A pre-built occupancy map (*house_explored.yaml* and *house_explored.pgm*) can be downloaded from the Turtle Navigation repository (https://github.com/jimas95/Turtle_Navigation/tree/main/turtle_slam/maps) and used for localisation. After launching RViz, the robot's initial pose can be set using the "2D Pose Estimate" tool until the robot aligns with the map and laser scan, as shown in Figure 2. The robot can then be teleoperated using `ros2 run turtlebot3_teleop teleop_keyboard`, and waypoint positions can be recorded via *ros2 topic echo /amcl_pose* for use in navigation and energy management tasks.

The acting behaviour must be implemented using a Behaviour Tree (BT) or FSM. You may use Sequence nodes for ordered execution, Selector nodes for priority handling, and a Blackboard for sharing state (e.g., occupancy, light status, and energy conditions), revisiting the tutorial examples.

After visiting a waypoint, you must update the planning problem to mark it as visited, clear any priority, and update occupancy, lighting, and energy states. You should also log the status of each waypoint to the terminal.

3 Appendix – additional information

3.1 Final submission instructions

When you are done, submit a single ZIP folder of your workspace named **FirstInitial_Surname_EE650_Project.zip**, containing all the code developed for this assignment.

The zipped folder must include these sub-directories:

- behavior_trees_xml/ – BT or FSM XML definitions/scripts
- config/ – Parameter and YAML configuration files
- include/ – Custom BT/FSM headers (if applicable)
- Maps/ – Occupancy pre-build maps
- launch/ – ROS2 launch files
- pddl/ – PDDL domain and problem files
- src/ – Source code (Python/C++) and ROS2 nodes
- CMakeLists.txt and package.xml – Build and package configuration files
- README.md – Contains declaration, file descriptions
- Images – Screenshots of BT, PDDL, Nav2 demos

Your .zip file must also contain a README file that includes all of the following:

- You must include the following declaration text (and it must be true!):

By submitting this assignment, I declare that this is all my own work and does not include the work of any other person (from my class, from the internet, or elsewhere) or GenAI output (except where specifically noted below)

- You must specify exactly which files in the submission you personally developed or modified. If any of the files were based on tutorial files, specify what modifications you made exactly.
- If you used GenAI outputs for any part of the assignment you must specify which parts and you must include the prompts that you used

You will also need to prepare a short .mp4 video showing the simulation being launched and showing the behaviour from start to finish of the simulation (or 2 minutes duration, whichever is shorter). There is a maximum file size of 100 MB for this video.

3.2 Instructions to setup environment:

Step 1: Setting up TurtleBot 3 in ROS 2 Simulation

export TURTLEBOT3_MODEL=burger (any model such as burger, waffle, or waffle_pi may be used)

Step 2: Install Required Packages:

```
sudo apt update
```

```
sudo apt install ros-humble-turtlebot3-gazebo ros-humble-turtlebot3
```

Step 3: Create ROS 2 Workspace

```
mkdir -p ~/Surname_StudentID_EE650_ws/src
```

```
cd ~/Surname_StudentID_EE650_ws/src
```

Step 4: Install TurtleBot3 packages

```
ros-humble-turtlebot3 \
```

```
ros-humble-turtlebot3-gazebo \
```

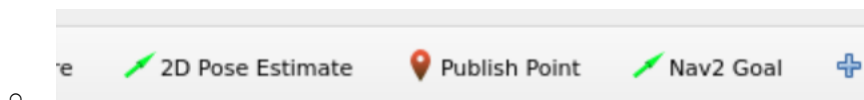
```
ros-humble-turtlebot3-navigation2 \
```

```
ros-humble-nav2-bringup
```

3.3 Hints for the overall system

Navigation Behaviour

- Use the NavigateToPose action (or Twist messages) for movement and obtain positions using `ros2 topic echo /amcl_pose`.
- Define (x, y) coordinates as waypoints, e.g., living_room [1.2, -0.5], kitchen [2.5, 1.0], bedroom [-1.5, 2.0], hallway [0.0, 0.0], entrance [-2.0, -1.0].
- Perform navigation by setting goals through the Nav2 Goal tool in Rviz, shown below.



Planning with PlanSys2

- Define a PDDL domain file (.pddl) including the actions you need
 - You may define different actions for waypoints with energy priorities, depending on your PDDL design. Your preconditions and effects must enforce that non-priority waypoints are not visited until all priority waypoints have been handled. Specifically, the plan must first handle the critical_energy waypoint, followed by the high_energy waypoint, and then all remaining waypoints.
 - You may define two predicates for each energy priority: one to represent whether a priority (e.g., critical_energy or high_energy) is active, and another to indicate the specific waypoint to which the priority is assigned.
- Define a problem using the plansys2 API (i.e. not via a PDDL problem file) specifying:
 - Initial robot position and waypoints.
 - Goal: all locations visited, all energy priorities cleared, and lighting/energy conditions appropriately handled.

Assignment – Smart Energy Management Robot

- Launch PlanSys2 with the domain and problem and allow it to generate the task plan using the default planner (POPF) or any other planner.

Useful Resources:

1. <https://github.com/rumair24/EE650/>
2. <https://py-trees-ros-tutorials.readthedocs.io/en/devel/>
3. https://plansys2.github.io/getting_started/index.html
4. <https://emanual.robotis.com/docs/en/platform/turtlebot3/simulation/>

Recommended Commands:

- `source /opt/ros/humble/setup.bash`
- `sudo apt update`
- `sudo apt install -y python3-colcon-common-extensions`
- `export LIBGL_ALWAYS_SOFTWARE=1`
- `export SVGA_VGPU10=0` (*improves Gazebo/RViz stability in WSL*)
- `use_sim_time:=True` (*uses simulation time instead of system time*)
- `map:=~/Surname_StudentID_EE650_Project/maps/house_explored.yaml` (*give correct path to occupancy map for Nav2 localisation*)